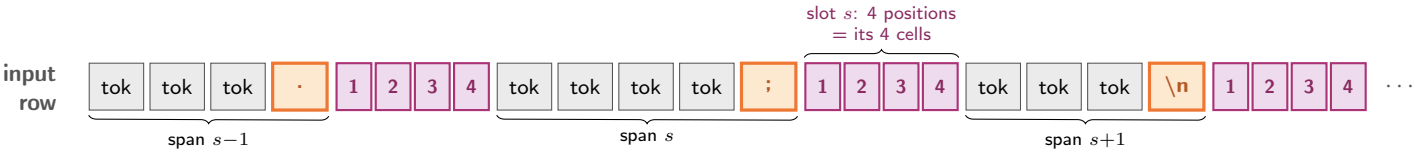


TUL = LXTUL + pointer + DITTO: the latent-selected slot loop

the 2026-10-07 winner · stage 1: morph/configs/lxtul_pointer.yaml, 5000 steps · stage 2: lxtul_pointer_ditto.yaml, 1000 steps (base.yaml still ships the older paid loop; this figure is the LXTUL recipe)

one row = 1024 tokens; after every span the packer inserts **4 slot positions** (≤ 64 slots, row length $1024 + 4 \cdot 64 = 1280$)
orange = boundary (., !?, newline, dashes; no comma) · span length 4–32 · every slot position's input = $E_{\text{slot}} + W_{\text{sent}} \text{ embed}(t_{\text{last}})$



STRICT GEOMETRY: inside the model, the slot loop is the only path from one span to a later span

stage	a token reads	a slot position / cell reads
prelude, 4 blocks	its own span's tokens only	its own span's tokens and its own earlier slot positions
slot loop, 6 blocks $\times T$	(tokens are not in the loop)	all 4 cells of its own slot + every cell of every earlier slot
coda, 4 blocks	its own span's tokens + the prefix positions of every earlier slot	itself only (it carries what the loop wrote, nothing else)

conv and value shift reset at every segment (retention is off); the coda's per-layer injections at slot positions are zeroed

PRELUDE (4 blocks): every position of the row, once

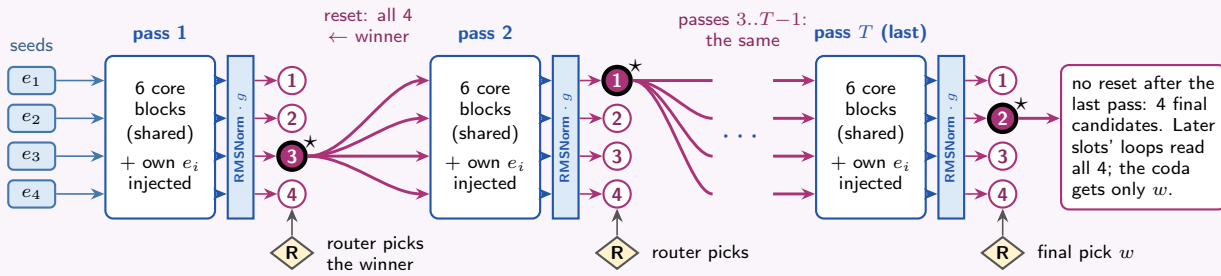
SEED the 4 cells of slot s (TULSlotRegister), $i = 1..4$:

$$e_i = \hat{x}_{s,i} + W_o \text{Attn}(Q_i; x_{j \in \text{span } s}) + P_{\text{cell}}[i]$$

$\hat{x}_{s,i}$ = normed prelude state at slot position i · Q_i = cell i 's own learned query, one head, keys and values = span s 's prelude token states
 W_o and P_{cell} start at zero · cell carrier = Hyper-Connection, 4 streams $\times d$ 1024 · $h_0 = e_i$

tokens: prelude state straight to the coda, never in the loop

THINK: every slot's 4 cells loop together; after each pass the router picks one cell and all 4 restart from it



legend: * = the pass's winner (dark fill) · = another cell · R = FanRouter, a low-rank scorer of each cell (its inputs are detached)
every pass: each cell gets its OWN seed again on the context channels, $h_{\text{ctx}} \leftarrow A h_{\text{ctx}} + dt e_{i,\text{ctx}}$ (DiagonalInjection, A starts at 0.447)
then RMSNorm with one gain g shared by all passes · $T \sim \text{Poisson}(6)$ per slot, clamped to 1..8, full BPTT
loop terms: epivol diversity on passes 1–2 (0.1), fixed-point term on the last pass (0.1)
gain hinge (100 at 0.9) on ONE pass per step; today the code draws the same pass on every step of a run (the draw restores the CPU RNG): an open bug

TRAIN ONLY: the router's teacher

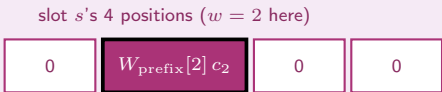
- an EMA twin of the prelude (momentum 0.996, no gradient) reads span $s+1$: $z_s = \text{LayerNorm}(\text{mean of its token states})$
 - head g (LayerNorm, MLP) reads the cells DETACHED; teacher pick:
 $\arg \min_i \|g(c_i) - z_s\|^2$
 - router: cross-entropy onto the teacher pick, at every pass
 - exit loss: relaxed winner-take-all on g (weight 1, ϵ 0.05) trains g only, so no latent gradient reaches the loop; a std floor on the online prelude's pooled span states (0.2)
- The forward follows the router, at train and at eval. z is a label only: nothing built from it enters the loop or the coda.

TRAIN ONLY: span decoder

2 layers, weight 1: teacher-forced decode of the NEXT span (span $s+1$). It reads the uniform MEAN of the 4 final candidates, not the winner.

WRITE: the winner alone

slot position $k \leftarrow W_{\text{prefix}}[k] c_k$ for $k = w$; the 3 other positions are EXACTLY 0 so the coda reads ONE live vector per earlier span

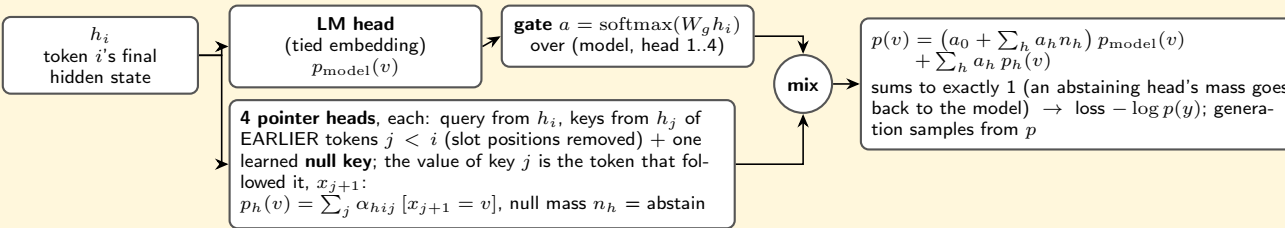


CODA (4 blocks): every position, once → final hidden state h_i (after the readout norm)

a token reads its own span + the prefix positions of every earlier slot · token-state dropout 0.15
no CE at slot positions; the boundary token keeps its ordinary label

h_i at every token position

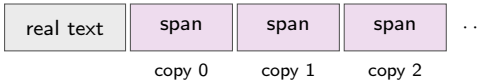
OUTPUT: the LM head mixed with an OUTPUT-ONLY POINTER HEAD (tul.pointer_heads: 4)



It writes nothing into any hidden state: inside the model the loop stays the only cross-span channel
the head only reads final states and copies tokens at the output · gate starts at (0.96, 0.01 per head) · never ternary

TRAINING STAGE 2: the DITTO phase (lxtul_pointer_ditto.yaml) against self-reinforcing repetition

1000 steps from the stage-1 checkpoint (weights only, fresh optimizer, 200-step LR ramp). 3 of each batch's 6 rows become **pseudo-repetition rows**: the real text up to a random span (never the first), then that span repeated to fill the row. The other 3 rows train as usual.



on every token of copy $n \geq 1$ the CE is replaced by $-\log(1 - |p_n - 0.5 \text{sg}(p_{n-1})|)$ (DITTO, Xu et al. 2022), with p the pointer-mixed probability of that token and p_{n-1} the same token one copy earlier (sg = stop-gradient)
mean over those positions, added to the CE mean at weight 1: each repeat must be about half as likely as the one before

MEASURED (MORPH, one 5090, 480 sweep rows, ONE seed per arm)

model	CE@6	gap to plain [95% CI]	K1–K6	seq_rep_4 at T 0.7, top-k 40
plain (no pointer)	4.0806	0		0.215
LXTUL	4.3529	+0.272	+0.0215	0.041 (degenerate text)
LXTUL + pointer, 5k	3.9280	−0.150 [−0.169, −0.132]	+0.0155	0.307
+ DITTO phase	trainer val +0.071 vs its control phase			0.140 (control phase 0.291, real text 0.020)

K1–K6 = CE at loop depth 1 minus CE at depth 6 · head forced off: CE 4.41 · owed: a MORPH plain + pointer ruler (on the Parcae testbed, plain + pointer led strict LXTUL + pointer by 0.045)
training speed: 11.8k tok/s, vs 22.2k for the Parcae testbed's LXTUL + pointer on the same rows (the MORPH step is host-serial: launch-bound, equal FLOPs)

lab/experiments/mixed/2026-10-06-lxtul-pointer.md · lab/experiments/failures/2026-10-07-lxtul-pointer-coverage.md · lab/experiments/mixed/2026-10-07-lxtul-pointer-ditto.md · lab/experiments/mixed/2026-10-07-morph-vs-parcae-speed.md

GENERATION

a new token costs its prelude pass and one coda pass, no core pass · an emitted boundary inserts 4 slot positions and costs ONE think for that new slot (T passes $\times 4$ cells)

today only the eager recompute generator (tul_generate.py) runs this model: the KV-cached generators refuse the fan (fan_k > 0) AND the pointer head (pointer_heads > 0), so decode speed is not yet measured